Procedimiento de respaldo y restauración · Plataforma Green Alliance

Cooperativa Green Alliance · NIT 902.103.335-7 · www.greenallianceco.com

Contrato Nº 001-2026 · Cláusula décima sexta («Copias de seguridad»), cláusula novena y Anexo Técnico Nº 1, §3.

Versión 1.0 · 2026-10-01

Este procedimiento informa a LA COOPERATIVA la **periodicidad, ubicación, conservación y restauración** de las copias de seguridad. Como pide la cláusula décima sexta, los respaldos del proveedor (Supabase) no reemplazan las copias propias en poder de la cooperativa.

1. Qué se respalda

Elemento	Dónde vive	Cómo se respalda
Código fuente	GitHub (`GreenAllianceCoo/PagGreenAlliance`)	El repositorio es el respaldo; además, una copia `.zip` de `main` en cada respaldo mensual
Estructura de la base de datos	Supabase Postgres + `supabase/migrations/`	`esquema.sql`
Datos (asociados, créditos, afiliaciones, comisiones, sorteos, usuarios de Auth)	Supabase Postgres	`datos.sql`
Roles de la base	Supabase Postgres	`roles.sql`
Fotos de afiliación (cédula y selfie)	Supabase Storage, bucket privado `afiliacion-documentos`	Copia de la carpeta del bucket
Logos de convenios	Supabase Storage, bucket público `convenios-logos`	Copia de la carpeta del bucket
Variables de entorno	Vercel	**Lista de nombres** (en `.env.example`). Los valores se guardan en el gestor de contraseñas de la cooperativa, nunca junto al respaldo

2. Respaldos automáticos del proveedor

Supabase hace respaldos automáticos diarios **solo en planes pagos** (Pro y superiores; la retención depende del plan y de si se contrata *Point-in-Time Recovery*). En el plan gratuito no hay respaldos

descargables.

PENDIENTE (Sebas / cooperativa): verificar en Supabase, **Project Settings** → **Billing**, qué plan tiene el proyecto `sqpmxizxkqorccpvjwoz`, y en **Database** → **Backups** cuántos días se conservan. Anotarlo aquí:

- Plan de Supabase: ____
- Respaldos automáticos: ☐ Sí, diarios, por ____ días ☐ No
- Point-in-Time Recovery: ☐ Sí ☐ No

Aunque existan, estos respaldos están dentro de la misma cuenta de Supabase: si se pierde el acceso a la cuenta, se pierden también. Por eso se hacen además los respaldos manuales de la sección 3.

3. Respaldo manual

3.1 Periodicidad

Cuándo	Qué incluye	Responsable
Cada semana (sugerido: lunes)	Base de datos (roles, esquema, datos)	Administrador técnico designado
Antes de cada migración o cambio en la base	Base de datos	Quien aplica la migración
Cada mes (primer lunes)	Base de datos + Storage + `.zip` del código de `main`	Administrador técnico designado
Al terminar el contrato y en el acta de entrega final	Todo lo anterior	EL CONTRATISTA, entregado a LA COOPERATIVA

3.2 Requisitos (una sola vez por equipo)

- Node.js 20 o superior y Git (Git Bash en Windows).
- **Docker Desktop instalado y abierto**: `supabase db dump` ejecuta `pg_dump` dentro de un contenedor de Docker. Sin Docker el respaldo falla.
- Una copia del repositorio y `npm install`.
- Acceso al proyecto de Supabase con una cuenta de la cooperativa y la contraseña de la base de datos (Supabase, **Project Settings** → **Database**). La contraseña se guarda en el gestor de contraseñas de la cooperativa.
- Enlazar el proyecto:
  `bash npx supabase login npx supabase link --project-ref sqpmxizxkqorccpvjwoz`

3.3 Con el script (recomendado)

Desde la raíz del repositorio:

```
# Semanal o antes de una migración: solo base de datos
bash scripts/respaldo.sh "D:/Respaldos Green Alliance" --motivo antes-migracion
```

```
# Mensual: base de datos + fotos y logos + .zip del código de main
bash scripts/respaldo.sh "D:/Respaldos Green Alliance" --con-storage --con-codigo --motivo mensual
```

El script crea una carpeta con fecha y hora (por ejemplo `2026-10-05-0900-mensual/`) con `roles.sql`, `esquema.sql`, `datos.sql`, `storage/` y `codigo-main.zip` si se pidieron, y `LEEME.txt` con el proyecto, el commit, la última migración y las sumas SHA-256 de cada archivo. Se niega a guardar dentro del repositorio y falla si algún archivo queda vacío.

3.4 A mano (los mismos comandos)

Así se hizo el respaldo del **2026-09-30** (carpeta `../backups/2026-09-30-antes-requerimientos`, junto al repositorio y fuera de él, antes de aplicar las migraciones de los requerimientos de la cooperativa):

```
npx supabase db dump --linked -f esquema.sql
npx supabase db dump --linked --data-only -f datos.sql
npx supabase db dump --linked --role-only -f roles.sql
```

El script agrega `--use-copy` a los datos, que hace el archivo más rápido de restaurar; sin esa opción el respaldo también es válido.

3.5 Storage (fotos y logos)

Con la CLI (función experimental de Supabase):

```
npx supabase storage cp -r ss:///afiliacion-documentos ./storage/afiliacion-documentos --linked --
npx supabase storage cp -r ss:///convenios-logos ./storage/convenios-logos --linked --experimental
```

Si la CLI falla, se descargan desde el panel de Supabase: **Storage → (bucket) → seleccionar carpetas → Download**.

Las fotos de afiliación de solicitudes ya resueltas pueden ser muchas. Si la cooperativa define un tiempo de conservación de esas fotos en su política de datos, aplica también a sus copias.

3.6 Después de cada respaldo

1. **Comprimir con contraseña** la carpeta (7-Zip, formato `.7z`, cifrado AES-256, con «Cifrar nombres de archivo»). El respaldo contiene cédulas, correos, cuentas de nómina y fotos de documentos (Ley 1581 de 2012).
2. Guardar la contraseña del archivo en el gestor de contraseñas de la cooperativa, no junto al archivo.
3. Copiar el `.7z` a las **dos ubicaciones** (sección 4).
4. Borrar la carpeta sin cifrar del equipo local.
5. Anotar el respaldo en el registro (sección 7).

4. Dónde se guardan

Siempre en cuentas **a nombre de LA COOPERATIVA** (cláusula décima), nunca en cuentas personales del contratista ni de empleados.

Ubicación	Ejemplo	Para qué
1. Nube de la cooperativa	Google Drive o OneDrive de la cuenta institucional, carpeta «Respaldos plataforma» con acceso solo para el representante legal y el administrador técnico	Copia principal, accesible desde cualquier lugar
2. Copia fuera de la nube	Disco externo o USB cifrado, guardado en la sede de la cooperativa	Por si se pierde el acceso a la cuenta de nube

PENDIENTE (Sebas / cooperativa): definir las dos ubicaciones concretas y quién tiene acceso:

- Ubicación 1: _____ ***Acceso:*** _____
- Ubicación 2: _____ ***Custodio:*** _____

5. Cuánto se conservan

Tipo	Se conservan
Semanales	Las últimas **8** (dos meses)
Antes de migración	**3 meses** desde la migración
Mensuales	**12 meses**
Respaldo de entrega final	Mientras la cooperativa lo considere necesario (mínimo durante la garantía de 30 días)

Al borrar respaldos vencidos, borrarlos de las dos ubicaciones. Si la política de tratamiento de datos de la cooperativa fija plazos distintos, prevalece la política.

6. Restauración

6.1 Restaurar en un proyecto nuevo (pérdida total o cambio de cuenta)

Se usa cuando el proyecto actual ya no existe o no se puede usar. Requiere el cliente de Postgres `psql` (viene con PostgreSQL; en Windows, el instalador de PostgreSQL con solo «Command Line Tools»).

1. **Crear el proyecto** en Supabase con la cuenta de la cooperativa, en la región más cercana (la actual es la que indique **Project Settings** → **General** del proyecto original). Guardar la contraseña de la base en el gestor de contraseñas.
2. **Descifrar** el respaldo elegido en una carpeta temporal y comprobar las sumas de `LEEME.txt`:
   `bash sha256sum -c <(sed -n '/SHA-256:/,$p' LEEME.txt | tail -n +2)`
3. **Cadena de conexión:** en el proyecto nuevo, **Connect** → **Session pooler** (o *Direct connection*), copiarla en la terminal sin guardarla en archivos:
   `bash export NUEVA_DB="postgresql://postgres.<ref>:<contraseña>@<host>:5432/postgres"`
4. **Aplicar roles, esquema y datos**, en ese orden y en una sola transacción:
   `bash psql --single-transaction --variable ON_ERROR_STOP=1 \ --file roles.sql \ --file`

`esquema.sql \ --command 'SET session_replication_role = replica' \ --file datos.sql \ --dbname "$NUEVA_DB"`

`session_replication_role = replica` evita que los triggers (bitácoras inmutables, validaciones) bloqueen la carga de datos históricos. Si `roles.sql` da error porque un rol ya existe, se puede quitar ese archivo del comando: los roles estándar de Supabase ya vienen en el proyecto nuevo.

5. **Storage:** crear los buckets si no los creó el esquema (`afiliacion-documentos` privado, `convenios-logos` público) y subir las carpetas:
   `bash npx supabase link --project-ref <ref nuevo> npx supabase storage cp -r ./storage/afiliacion-documentos ss:///afiliacion-documentos --linked --experimental npx supabase storage cp -r ./storage/convenios-logos ss:///convenios-logos --linked --experimental`
6. **Auth:** en el proyecto nuevo, **Authentication → URL Configuration**: *Site URL* `https://www.greenallianceco.com`. En **Authentication → Emails**, configurar el SMTP (Resend) y la plantilla *Magic Link* / código de ingreso (`supabase/templates/codigo-ingreso.html`), igual que en el proyecto original. En **Authentication → Providers → Email** dejar *Secure email change* activo (valor por defecto; no se pega plantilla de cambio de correo). Los usuarios y sus correos vienen en `datos.sql` (esquema `auth`).
7. **Vercel:** en **Settings → Environment Variables**, cambiar `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` y `SUPABASE_SERVICE_ROLE_KEY` por los del proyecto nuevo (**Project Settings → API**). Las demás variables no cambian. Luego **Deployments → Redeploy**.
8. **Migraciones pendientes:** si el respaldo es anterior a la última migración del repositorio, aplicar las que falten:
   `bash npx supabase migration list --linked npx supabase db push --linked`
   Si `migration list` muestra como pendientes migraciones que ya venían en el respaldo (el historial de migraciones puede no restaurarse), **no** ejecutar `db push`: marcarlas como aplicadas con `npx supabase migration repair --status applied <versión> --linked` y aplicar solo las realmente nuevas.
9. **Prueba** (sección 6.3) y borrar la carpeta descifrada.

6.2 Recuperar datos en el mismo proyecto

- **Error reciente con respaldos automáticos activos:** Supabase, **Database → Backups → Restore** (restaura todo el proyecto a ese punto; lo hecho después se pierde).
- **Recuperar una tabla o unos registros:** no restaurar encima de producción. Restaurar el respaldo en un proyecto temporal (6.1, pasos 1–4), copiar desde ahí solo lo necesario y borrar el proyecto temporal.

6.3 Prueba después de restaurar

- ☐ `https://www.greenallianceco.com` carga la landing y los convenios con sus logos.
- ☐ Un asociado de prueba ingresa con cédula y código, ve su estado y su carné.
- ☐ Un administrador ingresa a `/admin` y ve afiliaciones, asociados y créditos con el número esperado de registros.
- ☐ El detalle de una afiliación muestra las 3 fotos.
- ☐ `/verificar/<código>` de un carné responde.

- ☐ El conteo de filas de `perfiles`, `solicitudes_credito` y `solicitudes_afiliacion` coincide con el de antes (si se conoce).
- ☐ La tarea programada responde: en Vercel, **Settings** → **Cron Jobs**, ejecución sin error 401.

6.4 Simulacro

Al menos **una vez cada 6 meses** se restaura el último respaldo mensual en un proyecto temporal de Supabase (6.1, pasos 1–5) para comprobar que sirve. Se anota en el registro y se borra el proyecto temporal.

7. Registro de respaldos

Fecha	Tipo	Motivo	Ubicaciones	Hecho por	Verificado
2026-09-30	Base de datos (roles, esquema, datos)	Antes de los requerimientos de la cooperativa	Equipo del contratista (`backups/2026-09-30-antes-requerimientos`, fuera del repositorio). **PENDIENTE: cifrar y entregar a la cooperativa**	Sebastián Sandoval	Archivos no vacíos